

Figure 1: Android architecture

frameworks/base/services/java/com/android/server/ provides the path of the Java source files for these services. Most of these services have a JNI counterpart, through which these services talk to the lower layers like the kernel. In an Android phone, this list of services can be viewed by issuing the command *service list* through the *adb* shell.

System services start-up

After boot-up, the Linux kernel starts the *init* process, which is the grand-daddy of all processes in the system, with a PID = 1. The following steps explain the start-up procedure until the *Home* screen shows up:

1. A set of native services (written in C/C++) like *vold*, *netd*, *installd* and *debuggerd* are started. These services have root permission.
2. Then the *Service Manager* and *Media Server* are started. These run with the system's permission.
3. *init* then starts the *app_process* (*frameworks/base/cmds/app_process/app_main.cpp*). This launches the 'zygote' process, in a Dalvik VM via *frameworks/base/core/jni/AndroidRuntime.cpp* and *frameworks/base/core/java/com/android/internal/os/ZygoteInit.java*. This zygote process then becomes the parent of every application.
- a. When the zygote starts, it has an active instance of the Dalvik VM that contains the classes required by the applications pre-loaded in its address space. In Android, each application runs its own Dalvik VM. When an application is started, zygote 'forks' a child process (which is the app process); both the zygote and the app processes use the same VM instance until the child does a 'write' (modifies the data), in which case a copy of the Dalvik VM is created, for the child. This ensures a faster start-up time for the applications. Thus the zygote becomes the parent for all the applications.
- b. We can actually check this in the *adb* shell using the

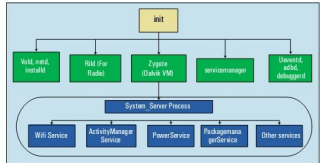


Figure 2: System services startup

CAUTION!

Most of the details that we have discussed here are not documented very well at developer.android.com. This article is my understanding of the system, gained from browsing through the source code. So feel free to send me clarifications or corrections.

Next month, we will discuss the skeleton of a typical system service.

'ps' commands.

'ps | grep zygote' and 'ps | grep com.android'

...will reveal this relationship.

4. The zygote starts the *system_server* process. This code is in the */init.rc* file in an Android phone:

```
service zygote /system/bin/app_process -Xzygote /system/bin
--zygote --start-system-server
```

5. This *system_server* process then starts all the system services. These services run as threads inside the *system_server* process. This can be verified by running the following command in the *adb* shell. This command lists the names of the threads running as part of the *system_server* process.

```
$: cat /proc/'pidof system_server'/task/*/comm
```

The system server code is in *frameworks/base/services/java/com/android/server/*, named as *System_server.java*.

6. One of the services is the *ActivityManagerService*, which as a final step of its initialisation, starts the 'Launcher' application (which brings up the *Home Screen*) and sends the *BOOT_COMPLETE Intent*. This intent signals to the rest of the platform that the system has completed boot up. **END.**

By: R Durgadoss

The author is a kernel programmer and spends his spare time hacking Android. You can reach him at r.durgadoss@gmail.com